

Projeto Final em Engenharia Informática

Musical Theory Trainer

MUSICAL THEORY STUDY COMPANION / COMPANHEIRO DE ESTUDO DE TEORIA
MUSICAL

RELATÓRIO FINAL

Daniel Junior - 2304335

Pedro Duarte Pestana

Junho 2026

[página intencionalmente em branco]

Índice

Índice.....	2
Lista de Figuras	5
Lista de Tabelas.....	6
Resumo.....	7
Capítulo 1 - Introdução	8
1.1 Descrição do Projeto	8
1.2 Necessidades Identificadas	8
1.3 Âmbito.....	8
1.4 Objetivos	9
1.5 Restrições	9
1.6 Resultados Esperados	10
1.7 Alterações face à Proposta Original	10
1.7.1 Nomenclatura dos tipos de exercício	10
1.7.2 Janela do algoritmo adaptativo: 10 -> 100 exercícios.....	11
1.7.3 Escala de dificuldade: 1-5 -> 1-10	11
1.7.4 Localização da lógica adaptativa: frontend -> backend	11
1.7.5 Refinamentos da API identificados em auditoria pré-intercalar.....	11
1.8 Calendário Atualizado	11
Capítulo 2 - Desenho	12
2.1 Stack Tecnológica	12
2.2 Arquitectura C4 - Nível 1 (Contexto).....	13
2.3 Arquitectura C4 - Nível 2 (Contentores)	14
2.4 Modelo de Dados	15
2.5 Algoritmos Principais.....	16
2.5.1 Dificuldade Adaptativa (RF09)	16
2.5.2 Identificação de Padrões Fracos (RF08)	17
2.5.3 Geração Procedural de Exercícios	18
2.6 Padrões de Design.....	20
Strategy (GoF)	20
Factory Method (GoF)	20
Data Access Object (Core J2EE Patterns).....	20
Value Object / Record (DDD)	20
Sentinel Value	20

2.7 Wireframes	20
2.8 Decisões de Arquitectura	21
ADR-001 - Arquitectura em 3 camadas	21
ADR-002 - Backend agnóstico à origem do input	21
ADR-003 - Geração procedural de exercícios sem datasets externos	21
ADR-004 - Web Audio API para reprodução de som	21
ADR-005 - Web MIDI API para input de hardware MIDI	21
ADR-006 - Java 21 + Spring Boot para o backend	21
ADR-007 - H2 para desenvolvimento; PostgreSQL ou SQLite3 para produção	22
ADR-008 - JavaScript vanilla no frontend (sem frameworks)	22
ADR-009 - Estrutura de ecrãs: páginas HTML separadas	22
ADR-010 - DAO classes com JDBC puro em vez de Spring Data JPA	22
ADR-011 - TDD como metodologia de desenvolvimento	22
ADR-012 - Modelo de dados e estratégia de persistência	22
ADR-013 - Schema do campo question nos exercícios	22
ADR-014 - Protocolo de resposta baseado em notas MIDI	22
ADR-015 - Taxonomia de dificuldade para exercícios	23
ADR-016 - Contrato de Erros: RFC 7807 Problem Details	23
ADR-017 - Frontend: localização no repositório e integração com Spring Boot	23
ADR-018 - Teclado virtual: range MIDI e design responsivo	23
ADR-019 - Expansão do domínio de escalas além do MVP original	23
ADR-020 - Testes de integração frontend: Cucumber + Selenide	23
ADR-021 - Enunciado de intervalo não revela o tipo	24
ADR-022 - Raízes de acorde limitadas a teclas brancas em BEGINNER e ELEMENTARY	24
ADR-023 - Presets de timbre Web Audio e scheduling nativo	24
Capítulo 3 - Implementação	24
3.1 Decomposição em Fases	24
3.2 Estado de Implementação à Data do Intercalar	25
3.3 Calendário da Fase Restante	25
Capítulo 4 - Testes	26
4.1 Filosofia e abordagem	26
4.2 Camadas de teste	26
Testes de unidade - domínio musical	26
Testes de integração - persistência	27
Testes baseados em propriedades - jqwik	27
Validação de qualidade dos testes	27

Testes E2E - Cucumber e Selenide	27
4.3 Métricas	27
4.4 O que os testes não cobrem (e porquê)	28
4.5 Cobertura da base de dados SQLite	28
4.6 Resultados de performance	28
Capítulo 5 - Conclusões	29
5.1 O que foi construído	29
5.1.1 Artefactos do projeto	29
5.1.2 Backend	29
5.1.3 Frontend	29
5.1.4 Milestones Concretas	29
5.2 Objetivos vs. resultado	29
5.2.1 Dificuldade Adaptativa	30
5.2.2 Added Value – Quality of Life Improvements	31
5.3 O que aprendi a fazer diferente	33
5.4 Limitações	33
5.5 Trabalho futuro	33
5.6 Nota final	34
Bibliografia	34

Lista de Figuras

Figura 1 - Diagrama de contexto C ₄ - utilizador, sistema e controlador MIDI externo.	14
Figura 2 - Diagrama de contentores C ₄ - frontend, backend e base de dados.....	15
Figura 3 - Diagrama entidade-relação - exercises, sessions e results.....	16

Lista de Tabelas

Tabela 1 - Tabela de Restrições.....	10
Tabela 2 - Calendário Atualizado.....	12
Tabela 3 - Stack Tecnológica	13
Tabela 4 - Decomposição em Fases.....	25
Tabela 5 - Calendário da Fase Restante	26
Tabela 6 - Estado final suite de testes.....	28
Tabela 7 - Resultado Testes Performance de Endpoints.....	29
Tabela 8 - Requisitos Funcionais	32
Tabela 9 - Requisitos Não Funcionais.....	32

Resumo

O Musical Theory Trainer é uma aplicação web de treino para teoria musical. O utilizador lê descrições, ouve intervalos, escalas e acordes e responde tocando as notas num teclado virtual ou num controlador MIDI físico. A aplicação avalia automaticamente, regista o progresso e adapta a dificuldade ao desempenho do utilizador ao longo das sessões. (Wozniak & Gorzelańczyk, 1994)

Este relatório intercalar documenta o desenho completo do sistema - arquitectura em 3 camadas (Fowler, 2002), modelo de dados , três algoritmos principais (dificuldade adaptativa, identificação de padrões fracos, geração procedural), wireframes e 19 ADRs - e o estado de implementação a 2 de maio de 2026: backend completo com 230 testes a passar, frontend especificado (Spring Boot 3.3.0 Reference Documentation, 2024) e agendado para as semanas 9-12.

Capítulo 1 - Introdução

1.1 DESCRIÇÃO DO PROJETO

O Musical Theory Trainer é uma aplicação web de treino auditivo para teoria musical.

O mote que despoletou o projeto foi simples: um Duolingo para teoria musical. Mas rapidamente percebi que a múltipla escolha não era adequada para o que queria construir - não treina realmente o ouvido, apenas o reconhecimento de opções.

O resultado foi um sistema onde o utilizador ouve uma estrutura musical - um intervalo, uma escala, um acorde - e responde tocando as notas num teclado virtual (ou num controlador MIDI físico). A aplicação avalia automaticamente e devolve feedback imediato. O funcionamento foi pensado para ser simples e sem atrito:

- arranca com um único comando
- corre em qualquer browser moderno sem instalação adicional
- uma sessão pode ser completada em menos de 5 minutos
- não há login, não há subscrição, não há uma gamificação forçada

Há apenas o exercício e o feedback.

1.2 NECESSIDADES IDENTIFICADAS

Aprender teoria musical é um obstáculo frequente para músicos autodidactas e estudantes em turmas numerosas. (Bloom, 1984) O problema não é a falta de recursos - temos livros, vídeos, aplicações, etc. O problema é a falta de uma verificação personalizada que sirva para construir a segurança e a auto-estima do estudante. (Vygotsky, 1978)

Num mundo perfeito, este feedback seria imediato e ajudaria diretamente o desenvolvimento de competências musicais. (Bloom, 1984) Confirmar se um intervalo é mesmo uma 5ª Perfeita, ou se um acorde é menor, precisa de alguém de fora que conheça esses conceitos - normalmente um professor. (Bloom, 1984) Que raramente está disponível em tempo real durante a prática de exploração musical, aquela que realmente reforça o aprendizado e expande o conhecimento. Refiro-me ao momento privado e informal, sem o rigor da prática musical observada.

O resultado é que conceitos fundamentais ficam imprecisos durante anos. (Ebbinghaus, 1964) Esta imprecisão limita a capacidade de improvisar, ler partituras e comunicar musicalmente. É como aprender a andar de bicicleta com as rodinhas sempre colocadas: a insegurança leva à pouca exploração prática, e práticas repetitivas e mecânicas acabam por ser infrutíferas. (Csikszentmihalyi, 1990)

1.3 ÂMBITO

Dentro do âmbito:

- Exercícios de identificação de intervalos, escalas (Maior, Menor Natural, Menor Harmónica em níveis iniciais; modos diatónicos e escalas adicionais desbloqueados progressivamente com o nível de dificuldade) e tríades (maior, menor, diminuto, aumentado)
- Teclado virtual clicável com reprodução de som via Web Audio API (Web Audio API — W3C Recommendation, 2021)
- Suporte a controlador MIDI físico USB, com detecção automática (Web MIDI API)

- Avaliação automática com feedback imediato (correto/errado + resposta correta se errado)
- Sessões de treino com persistência de resultados entre utilizações
- Dashboard de progresso por tipo de exercício e identificação de padrões de erro
- Dificuldade adaptativa baseada nos últimos 100 exercícios por tipo
- Modo sandbox: tocar livremente e ver o nome das notas e intervalos em tempo real

Fora do âmbito (esta versão):

- Autenticação ou gestão de múltiplos utilizadores (CBo1, CBo2)
- Ditado rítmico (CBo5)
- Versão mobile nativa - aplicação web exclusivamente (CBo6)
- Progressões de acordes - reservadas para versão futura
- Reconhecimento de áudio por microfone

1.4 OBJETIVOS

Os critérios de aceitação observáveis do MVP estão organizados em nove objetivos:

- **Fo1** - Modelo de domínio musical amplo (Evans, 2003), mas não exaustivo: Note, Interval, Scale, Chord, com todos os casos verificados por testes unitários
- **Fo2** - Geração procedural de exercícios dos três tipos (intervalos, escalas, acordes), sem repetição consecutiva (mesmo exercício) na mesma sessão (Gamma, Helm, Johnson, & Vlissides, 1994)
- **Fo3** - Teclado virtual *clicável* no browser (C3 a C5 em mobile, C2 a C6 em desktop), com reprodução de som via Web Audio API (Web Audio API — W3C Recommendation, 2021)
- **Fo4** - Input via controlador MIDI físico com detecção automática, sem configuração manual (Web MIDI API — W3C Editor's Draft, 2015)
- **Fo5** - Avaliação automática com feedback imediato: correto/errado e resposta correta quando errado, em menos de 200ms em ambiente local
- **Fo6** - Feedback sonoro distinto para acerto e para erro
- **Fo7** - Persistência de sessões e histórico de resultados entre reinícios da aplicação
- **Fo8** - Dashboard de progresso com taxa de acerto por tipo de exercício e identificação dos padrões onde o utilizador tem pior desempenho
- **Fo9** - Dificuldade adaptativa: incrementa se taxa de acerto $\geq 80\%$, decrementa se $< 40\%$, janela de 100 exercícios por tipo para evitar oscilações por variância em amostras pequenas (Elo, 1978) (Wozniak & Gorzelańczyk, 1994)

1.5 RESTRIÇÕES

As principais restrições que condicionaram as decisões de design e implementação:

ID	Restrição
CB01	Sem autenticação nem login
CB02	Base de dados para um único utilizador implícito
CB05	Sem ditado rítmico - fora do âmbito desta versão
CB06	Aplicação web; sem versão mobile nativa
CB07	Frontend em HTML e JavaScript ES6+ vanilla - sem frameworks
CB11	Design visual não é prioridade - interface funcional e clara
RNF01	Backend: Java 21 + Spring Boot (Spring Boot 3.3.0 Reference Documentation, 2024), JDBC puro sem ORM
RNF06	Tempo de resposta da validação < 200ms em ambiente local

Tabela 1 - Tabela de Restrições

Para a lista completa de restrições e constraints, ver `docs/scope/requirements.md`.

1.6 RESULTADOS ESPERADOS

O resultado esperado é uma aplicação web funcional que arranca com um único comando (`mvn spring-boot:run`), corre no browser sem instalação adicional, e permite a qualquer utilizador completar uma sessão de exercícios de teoria musical em menos de 5 minutos.

O sucesso é verificável: se a aplicação gera exercícios, valida respostas corretamente, toca som, aceita input MIDI e regista progresso - o objetivo está atingido.

À data deste relatório intercalar (2 de maio de 2026), o backend está implementado e revisto: 230 testes a passar, todos os requisitos Must-have verificados. Na semana 8 foi realizada uma auditoria abrangente do backend contra os contratos documentados nos ADRs e requisitos, que identificou e corrigiu inconsistências - entre elas o facto de o gerador de escalas ignorar o sistema de dificuldade e a presença de um campo de múltipla escolha na API que contradizia o protocolo de resposta por notas MIDI. O frontend foi desenhado e especificado

- wireframes produzidos (ver `docs/design/wireframes.pdf`), decisões de arquitetura documentadas nos ADRs 017 e 018 - e a implementação está agendada apra as semanas 9 a 12.

A API REST (RFC 7807 - Problem Details for HTTP APIs, 2016) está documentada e testável interactivamente via Swagger UI, disponível em `http://localhost:8080/swagger-ui.html` após arranque da aplicação. Esta interface foi usada ao longo do desenvolvimento para verificação manual dos endpoints e foi o principal instrumento da auditoria realizada na semana 8.

1.7 ALTERAÇÕES FACE À PROPOSTA ORIGINAL

Durante a implementação, alguns parâmetros da proposta foram revistos. Documento-os aqui com a justificação respetiva.

1.7.1 Nomenclatura dos tipos de exercício

Os tipos `SCALE_IDENTIFICATION` e `CHORD_IDENTIFICATION` foram simplificados para `SCALE` e `CHORD` durante a implementação do modelo de domínio. O sufixo `_IDENTIFICATION` era redundante - todos os exercícios são de identificação. O critério de aceitação permanece o mesmo; apenas o valor do campo `type` na API mudou.

1.7.2 Janela do algoritmo adaptativo: 10 -> 100 exercícios

A proposta original previa uma janela de 10 exercícios para calcular a taxa de acerto. Optei por alargar para 100 porque, com apenas 10 amostras, uma sequência de 3 acertos consecutivos podia inflar artificialmente a dificuldade - provocando oscilações não representativas do nível real do utilizador. 100 exercícios por tipo fornecem uma taxa de acerto estatisticamente mais significativa e reduzem a variância aleatória. Ver ADR-015.

1.7.3 Escala de dificuldade: 1-5 -> 1-10

Ampliei a escala de dificuldade de 1-5 para 1-10. Convinceu-me a granularidade mais fina: +1/-1 num espaço de 10 é uma progressão mais gradual do que num espaço de 5. Além disso, a biblioteca alargada de tipos de escala (28 valores) precisava de mais bandas para uma classificação com significado pedagógico. Ficaram 5 bandas semânticas: BEGINNER, ELEMENTARY, INTERMEDIATE, ADVANCED, EXPERT. Ver ADR-015 para a taxonomia completa.

1.7.4 Localização da lógica adaptativa: frontend -> backend

A proposta original colocava a lógica de adaptação de dificuldade no frontend, com estado local dos últimos resultados. Movi-a para o backend. Penso que faz mais sentido centralizar a lógica de negócio no servidor:

- garante consistência entre sessões
- elimina a possibilidade de estado local corrompido no browser
- evita que a lógica seja contornada client-side

O `DifficultyService` implementa o algoritmo; o frontend recebe `suggestedDifficulty` na resposta de `generate` como sugestão informativa.

1.7.5 Refinamentos da API identificados em auditoria pré-intercalar

Na semana 8, antes de redigir este relatório, foi feita uma auditoria abrangente do backend contra os contratos documentados. Dois refinamentos relevantes resultaram desta revisão: O campo `options` foi removido da resposta de geração de exercícios. Este campo era um resquício do design original de múltipla escolha e contradizia o protocolo de resposta baseado em notas MIDI definido na arquitectura - o utilizador toca as notas, não selecciona de uma lista. A sua presença era uma inconsistência entre o código e o contrato documentado.

O domínio de escalas foi expandido para além das três escalas do MVP original (Maior, Menor Natural, Menor Harmónica). O enum `ScaleType` classifica 28 tipos de escala em bandas de dificuldade; o gerador passou a usar esta classificação correctamente, desbloqueando modos diatónicos (Dórico, Frígio, Lídio, etc.) a partir do nível ADVANCED e escalas exóticas no nível EXPERT. Esta expansão foi documentada depois, no ADR-019, durante a auditoria da semana 8.

1.8 CALENDÁRIO ATUALIZADO

O calendário original foi seguido com uma exceção: a semana 5 teve produtividade reduzida por sobreposição com a Páscoa e a entrega de eFólios de outras unidades curriculares (risco Ro4, identificado na proposta). O backend foi concluído na semana 8, dentro do prazo previsto para o intercalar.

Semanas	Datas	Realizado / Planeado	Marco
Sem. 1-2	17-28 mar	Kick-off com orientador. Proposta entregue: sinopse, MVP com critérios de aceitação, stack, calendário.	☑ Proposta (25 mar)
Sem. 3-4	31 mar-11 abr	Levantamento de requisitos MoSCoW (RF01-RF14, RNF, CB, OI). ADRs 001-011 formalizados. Diagramas C4 e modelo de dados. Repositório GitHub configurado.	-
Sem. 5	14-18 abr	Produtividade reduzida - Páscoa e eFólios concorrentes (risco Ro4).	-
Sem. 6	21-25 abr	Fase 2 - Persistência: DTOs, DAOs JDBC, 24 testes de integração. Suporte multi-base-de-dados. ADR-012.	-
Sem. 7	28 abr-2 mai	Fase 3-4 - Geradores de exercícios e REST API completa. 171 testes. ADR-013, ADR-014. Demo interna ao orientador não ocorreu. Estou atrasado e não contactei o orientador. RF07 (persistência), RF08 (dashboard), RF09 (dificuldade adaptativa). Auditoria abrangente do backend e correcção de inconsistências. Backend revisto. 230 testes.	✗ Demo interna
Sem. 8	3-6 mai	Relatório intercalar. Testes funcionais com Swagger para garantir funcionamento antes de começar implementação do UI. Tentar agendar demo interna.	Intercalar (6 mai)
Sem. 9-10	7-16 mai	Fase 5-6 - Frontend: teclado virtual, Web Audio API, Web MIDI API, ecrã de exercício.	-
Sem. 11-12	19-30 mai	Fase 7 - Dashboard de progresso, ecrã de fim de sessão. Testes de integração.	-
Sem. 13	2-6 jun	Revisão geral. Validação de todos os critérios de aceitação. Capturas de ecrã para Cap. 4.	-
Sem. 14	9-13 jun	Cap. 4 (Testes) e Cap. 5 (Conclusões). Revisão bibliográfica APA. Anexos.	-
Sem. 15	16-20 jun	Reunião de preparação para defesa. Ensaio de perguntas de júri.	Prep. defesa
Sem. 16	24 jun	Submissão do relatório final. Código e demo linkados no repositório.	Final (24 jun)

Tabela 2 - Calendário Atualizado

Capítulo 2 - Desenho

2.1 STACK TECNOLÓGICA

A selecção tecnológica foi orientada por dois princípios: reduzir dependências externas ao mínimo e usar tecnologias onde já tinha experiência prévia.

Componente	Tecnologia	Justificação
Backend	Java 21 + Spring Boot 3.3	Experiência prévia; Spring Boot minimiza configuração de infraestrutura (Spring Boot 3.3.0 Reference Documentation, 2024); JDBC puro sem ORM (Alur, Crupi, & Malks, 2003) mantém controlo explícito das queries SQL (ADR-010)
Frontend	HTML + JavaScript ES6+ vanilla	Sem experiência prévia com frameworks - React ou Vue adicionariam curva de aprendizagem desproporcional para o scope do projeto (ADR-008)
Áudio	Web Audio API (nativa)	Zero dependências externas; funciona em qualquer browser moderno sem instalação adicional; suficiente para sintetizar notas isoladas (ADR-004) (Web Audio API — W3C Recommendation, 2021)
MIDI	Web MIDI API (nativa)	Zero dependências; detecção automática de controladores USB; o backend é agnóstico à origem da nota - MIDI ou teclado virtual produzem o mesmo tipo de resposta (ADR-005) (Web MIDI API — W3C Editor's Draft, 2015)
Base de dados (dev)	H2 in-memory	Arranca sem configuração; schema recriado a cada reinício; consola web integrada para inspecção durante desenvolvimento (ADR-007)
Base de dados (prod)	SQLite3	Zero configuração de servidor, ficheiro único portátil, adequado para aplicação single-user; decidido em OI01 (ADR-001)
Build	Maven	Standard no ecossistema Java/Spring; gestão de dependências madura e estável

Tabela 3 - Stack Tecnológica

2.2 ARQUITECTURA C4 - NÍVEL 1 (CONTEXTO)

O sistema Musical Theory Trainer é uma aplicação web standalone. Não há sistemas externos de autenticação, APIs de terceiros nem serviços de armazenamento remoto. A única dependência externa opcional é um controlador MIDI físico ligado por USB, cujos eventos são capturados directamente pelo browser via Web MIDI API. (Web MIDI API — W3C Editor's Draft, 2015)

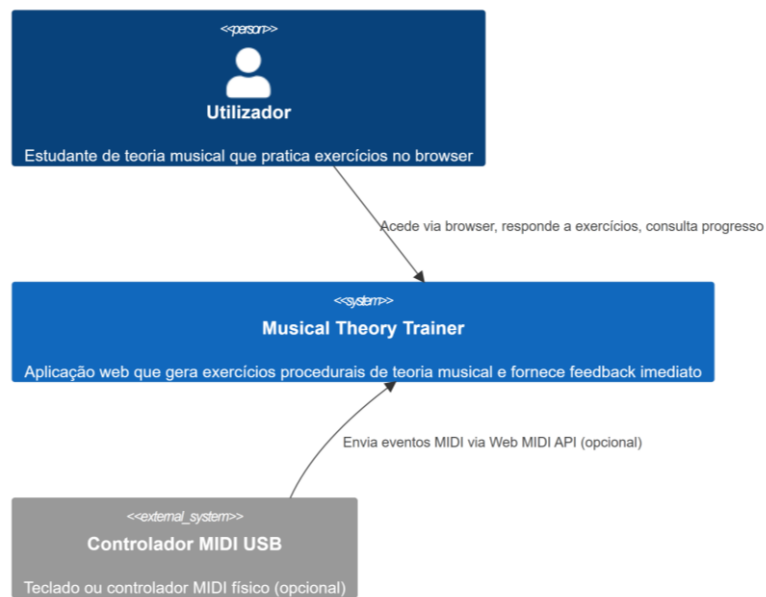


Figura 1 - Diagrama de contexto C₄ - utilizador, sistema e controlador MIDI externo.

2.3 ARQUITECTURA C₄ - NÍVEL 2 (CONTENTORES)

A aplicação divide-se em três contentores: o frontend (HTML + JS vanilla), o backend (Spring Boot) e a base de dados (H2 em desenvolvimento, SQLite3 em produção). O frontend comunica com o backend exclusivamente via REST/JSON. O backend persiste dados via JDBC/SQL. O frontend é servido pelo mesmo processo Spring Boot, a partir da directoria `/frontend/` configurada em `application.properties` (ADR-017).

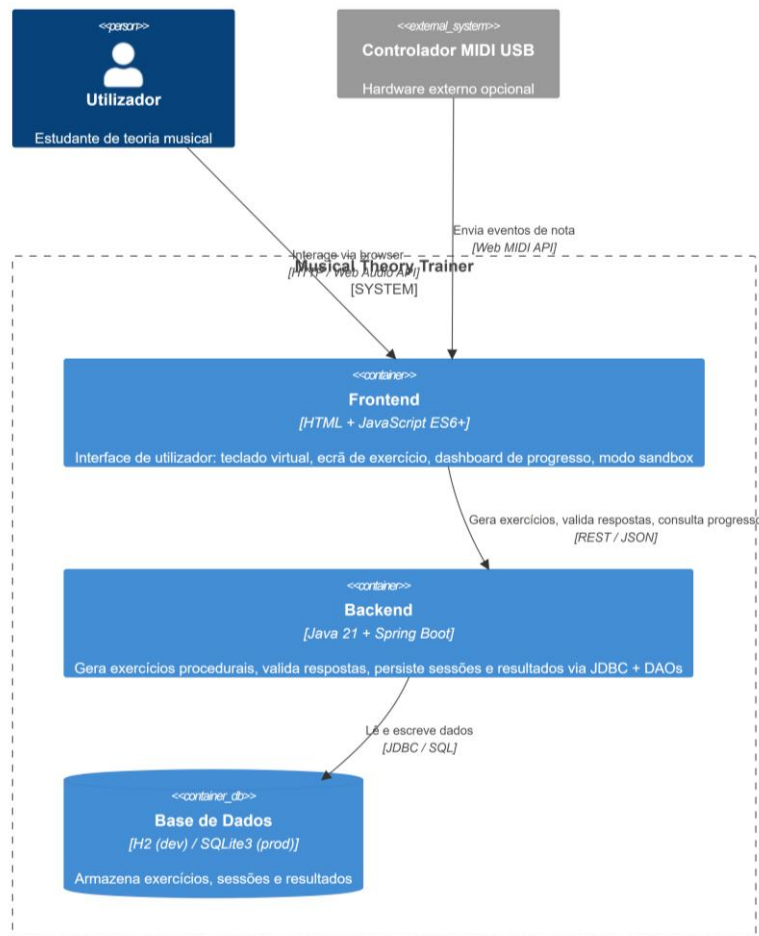


Figura 2 - Diagrama de contentores C₄ - frontend, backend e base de dados.

2.4 MODELO DE DADOS

O modelo de dados tem três tabelas: `exercises`, `sessions` e `results`.

- `exercises` - regista cada exercício gerado: tipo (INTERVAL, SCALE, CHORD), dificuldade (1-10), `question` (JSON com dados da pergunta, schema em ADR-013) e `correct_answer` (array de notas MIDI esperadas em JSON)
- `sessions` - ciclo de vida de uma sessão de treino: `start_time`, `end_time` (null se activa), contadores atómicos `total_exercises`, `correct_answers`, `incorrect_answers`
- `results` - regista cada resposta: FK para sessão e exercício, `user_answer` (notas MIDI tocadas em JSON), `is_correct`

A FK `results.session_id` usa ON DELETE CASCADE - apagada a sessão, os resultados são apagados.

Quando `session_id = 0` (SESSION_NONE, ADR-014), a resposta é avaliada mas não persistida - modo sandbox.

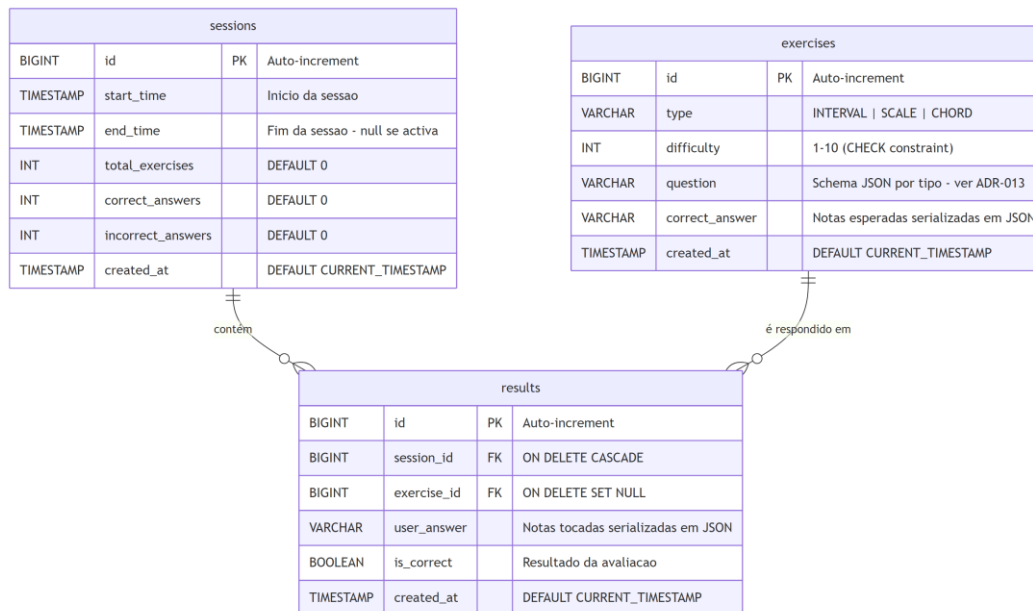


Figura 3 - Diagrama entidade-relação - exercises, sessions e results.

2.5 ALGORITMOS PRINCIPAIS

A aplicação implementa três algoritmos centrais: adaptação de dificuldade, identificação de padrões fracos e geração procedural de exercícios. Nenhum deles é sofisticado - o objectivo foi que fossem simples, justificáveis e suficientes para o scopedo projeto.

2.5.1 Dificuldade Adaptativa (RF09)

Contexto pedagógico

Enquanto pensava sobre o sistema de dificuldade, o problema ficou claro rapidamente: se a dificuldade ficar muito alta, o utilizador desiste; se ficar muito baixa, os exercícios tornam-se chatos e desinteressantes. Fui pesquisar estratégias para equilibrar isto - com o número de recursos pedagógicos e jogos existentes, alguém, de certeza, já pensou sobre o tema.

Nas minhas pesquisas, deparei-me com o princípio do Goldilocks (Csikszentmihalyi, 1990): a ideia de que existe uma zona ótima de desafio onde uma tarefa é suficientemente difícil para ser estimulante, mas alcançável o suficiente para não causar frustração e abandono. O conceito tem raízes na teoria do flow - o estado de absorção total (Csikszentmihalyi, 1990) numa atividade onde as capacidades do utilizador e o nível de desafio estão alinhados.

No treino auditivo esta zona é especialmente estreita. A diferença entre "consigo identificar este intervalo com esforço" e "é sempre o mesmo erro" é pequena, mas decisiva para manter a prática. A dificuldade adaptativa existe para manter o utilizador nessa zona, ajustando automaticamente com base no histórico recente, sem intervenção manual.

Alternativas consideradas

Considerei três abordagens antes de optar pela atual:

- **Dificuldade fixa:** simples de implementar, mas sem qualquer personalização. Dois utilizadores com níveis muito diferentes teriam a mesma experiência. Rejeitada.
- **Sistema Elo** (como no xadrez) (Elo, 1978): teoricamente elegante, mas desenhado para contextos competitivos com dois oponentes. Adaptá-lo para exercícios de treino individual implicaria definir um

"opponente" artificial e calibrar o K-fator para música. A complexidade não era justificada para um MVP.

- Spaced Repetition / SRS (como no Anki) (Ebbinghaus, 1964) (Wozniak & Gorzelańczyk, 1994): otimizado para memorização de factos estáticos. Teoria musical requer construção de competência auditiva, não apenas memorização de flashcards. A curva de repetição espaçada não mapeia naturalmente para "quão bem tocas um acorde diminuto".

Optei por uma abordagem de limiar simples sobre uma janela deslizante. É transparente, fácil de testar e fácil de explicar ao utilizador.

Justificação dos parâmetros

- **80% para subir, 40% para descer:** o limiar de 80% aproxima-se da "zona de desenvolvimento proximal" (Vygotsky, 1978) - um nível onde o utilizador consegue resolver a maioria dos problemas com algum esforço, mas não de forma automática. Abaixo de 40% o utilizador está claramente a falhar mais do que a acertar, o que é contraproducente pedagogicamente. A assimetria (80 vs 40) é intencional: é mais fácil descer do que subir, o que reduz a frustração.
- **Janela de 100 exercícios:** com uma janela de 10 (como previa a proposta original), três acertos consecutivos produziam 100% de acerto e disparavam uma subida imediata. Isso não reflecte o nível real - é ruído estatístico. Com 100 exercícios, 80% implica 80 acertos verificados, uma amostra genuinamente representativa.
- **Pinçamento (Clamping) +-2 sobre a sugestão:** sem este clamp, um utilizador a praticar em dificuldade 1 que o sistema considera deveria estar em 7 seria forçado para 5 imediatamente. O clamp preserva a intenção do utilizador dentro de um intervalo razoável. Se o utilizador pede dificuldade 1, o sistema pode sugerir 3 no máximo, não 7.
- **Backend em vez de frontend:** a proposta original colocava esta lógica no frontend. Movi-a para o backend por três razões: consistência entre sessões (o estado não se perde ao fechar o browser), impossibilidade de manipulação client-side, e centralização da lógica de negócio no servidor, onde acredito que esta pertence.

Pseudocódigo

```
entrada: tipo T, dificuldade pedida D
1. taxa = (acertos nos últimos 100 resultados com exercises.type = T) / total
2. sugestão = D+1 se taxa >= 0.80 | D-1 se taxa < 0.40 | D caso contrário
3. sugestão = clamp(sugestão, 1, 10)
4. effectiveDiff = clamp(D, sugestão-2, sugestão+2)
5. retornar effectiveDiff ao gerador e sugestão informativa ao frontend
```

Limitações conhecidas

O algoritmo trata todos os tipos de exercício como independentes. Melhorar em intervalos não afeta a dificuldade de escalas. Isso é correto para o MVP, mas num sistema mais sofisticado haveria transferência de aprendizagem entre tipos relacionados. Optei por não adicionar mais funcionalidades relacionadas à transferência de aprendizagem neste MVP.

2.5.2 Identificação de Padrões Fracos (RFo8)

Contexto pedagógico

Saber que tenho 65% de acerto em intervalos não me diz o suficiente para melhorar. Saber que tenho 20% de acerto no trítono específico, sim. A granularidade da identificação de fraquezas é o que torna o feedback útil - a dica pedagógica que a aplicação fornece deve ser específica o suficiente para orientar a prática.

Alternativas consideradas

- **Agregação apenas por tipo de exercício:** registrar "intervalos: 65%", "escalas: 70%". Demasiado grosseiro - não distingue onde dentro de cada tipo o utilizador falha.
- análise por machine learning / clustering: identificar padrões automaticamente sem regras definidas à priori. Seria academicamente interessante mas completamente fora de scope para um MVP, e requereria volume de dados que um utilizador individual nunca produziria em tempo útil.
- **Tags manuais pelo professor:** em aplicações como o Teoria existe marcação manual de dificuldade. Não se aplica aqui a este MVP - não há professor, é auto-serviço (*self-service*). Escolhi um método de agregação SQL simples, por (tipo, padrão específico) e ordenação pela taxa de acerto. É direto, sem dependências externas, e produz o nível de granularidade necessário.

Justificação dos parâmetros

- **Mínimo de 3 tentativas:** com 1 ou 2 tentativas, um único erro produz 0% de acerto
- estatisticamente irrelevante. 3 é o mínimo para que a taxa de acerto seja uma medida com algum significado. Há aqui um compromisso: com poucos dados, algumas fraquezas reais ficam fora da análise. Aceito essa limitação em favor de não mostrar falsos positivos.
- **LIMIT 10:** mostrar as 10 áreas com pior desempenho. Na prática, o utilizador não consegue focar em mais do que 2-3 áreas de melhoria em simultâneo. 10 é um limite superior conservador; a apresentação no dashboard pode filtrar mais.
- **`questionJson` como chave de agrupamento:** para intervalos, o grupo é o intervalo específico (trítono, 5ª Perfeita, etc.), não apenas "intervalos em geral". Para escalas e acordes, o grupo é o tipo (DORIAN, DIMINISHED, etc.). Esta granularidade é o que considero ter valor pedagógico real.

Pseudocódigo

1. `SELECT type, questionJson, COUNT(*) total, SUM(is_correct) acertos
FROM results JOIN exercises ON exercise_id = id
GROUP BY type, questionJson
HAVING COUNT(*) >= 3
ORDER BY acertos * 1.0 / COUNT(*) ASC
LIMIT 10`
2. para cada linha: extrair padrão do questionJson
`INTERVAL → fromSemitones(|notes[1]-notes[0]|).internalName()
SCALE / CHORD → questionJson.type`
3. `WeaknessHintProvider.getHint(type, padrão) → dica pedagógica para o utilizador`

Limitações conhecidas

O algoritmo não distingue entre "sempre errei este padrão" e "errei muito este padrão recentemente". Uma fraqueza antiga já superada pode continuar a aparecer se houver histórico suficiente. Uma solução futura seria aplicar um fator de decaimento temporal.

2.5.3 Geração Procedural de Exercícios

Contexto pedagógico

Um conjunto fixo de exercícios pode ser memorizado. Após algumas repetições, o utilizador reconhece o exercício pela sua "aparência" antes de o ouvir, o que invalida o treino auditivo. A geração procedural garante que cada sessão é genuinamente diferente, forçando o reconhecimento a partir do som e não da memória

visual. Curiosamente, o que queremos obter é o efeito oposto: a memorização de conceitos musicais, porém recorrendo a esta aparente imprevisibilidade por meio da aleatoriedade, penso que estamos a mascarar o objetivo final por meio do "reconhecimento" visual, auditivo e "matemático" escondido sob as bestes de teoria.

Alternativas consideradas

- **banco de exercícios estático:** simples de implementar, mas finito e memorizável. Aplicações como o Ear Trainer da Teoria usam bancos curados - funcionam para conteúdo de alta qualidade, mas limitam a variedade. Exigiriam também um sistema próprio para manutenção, em caso de adições, correções ou remoções de exercícios.
- **Conteúdo via API externa:** delegar a geração a um serviço externo. Rejeitado por criar dependência externa, latência, impossibilidade de uso offline, e questões de direitos de autor.
- **Exercícios gerados por LLM:** tecnicamente possível, porém não determinístico - um LLM pode "inventar" intervalos ou escalas incorretas. Teoria musical é uma disciplina estabelecida, não tolera erros factuais; um intervalo de "3.5 semitons" não existe .
- Optei por construir os exercícios diretamente a partir do modelo de domínio Java - os mesmos objetos `Scale`, `Chord`, `Interval` que definem as regras musicais. A correção de um exercício gerado é garantida pelo modelo, não por validação externa.

Justificação das escolhas de design

- **Notas brancas para iniciantes:** para utilizadores `BEGINNER` e `ELEMENTARY`, as raízes são restritas às notas brancas do piano (C, D, E, F, G, A, B). Preto implica notação enarmónica (C# = Db) que introduz complexidade conceptual antes do momento certo. A música tonal básica - que é onde começa o treino auditivo - usa predominantemente notas brancas.
- **Filtragem de aliases (`isAlias()`):** o enum `ScaleType` contém `IONIAN` como alias de `MAJOR`. Sem filtro, o gerador produziria exercícios "de escala `IONIAN`" e "de escala `MAJOR`" que são musicalmente idênticos, confundindo o utilizador sem valor pedagógico adicionado.
- **Seleção aleatória dentro da banda:** não sequencial, não ponderada. A aleatoriedade garante que o utilizador não antecipa o próximo exercício, o que é o requisito central do treino auditivo.

Pseudocódigo

```
entrada: tipo T, dificuldade D
1. banda = DifficultyLevel.of(D)
2. disponíveis = T.availableFor(banda).filter(!isAlias)
3. tipo_escolhido = random(disponíveis)
4. raiz = MIDI aleatório por banda
   se banda <= ELEMENTARY: raiz ∈ notas brancas (C3-B4)
   senão: raiz ∈ [C2, C5] cromático
5. objecto = domain.get(tipo_escolhido, Note.fromMidi(raiz))
6. notesToPlay = objecto.getNotes() + oitava_final se escala
7. guardar em BD; devolver ao frontend
```

Limitações conhecidas

A aleatoriedade uniforme dentro de uma banda significa que um iniciante pode receber `PENTATONIC_MAJOR` e `MINOR_NATURAL` com igual probabilidade, mesmo que `MAJOR` seja mais fundamental pedagogicamente. Uma versão futura poderia introduzir pesos por frequência de uso no ensino tradicional.

2.6 PADRÕES DE DESIGN

Strategy (GoF)

`ExerciseGenerator` é uma interface que define o contrato de geração e reconstrução de exercícios. `IntervalExerciseGenerator`, `ScaleExerciseGenerator` e `ChordExerciseGenerator` implementam a interface independentemente. Adicionar um novo tipo de exercício não requer alterar `GeneratorFactory` nem `ExerciseService`. (Gamma, Helm, Johnson, & Vlissides, 1994), pp. 315-323.

Factory Method (GoF)

`GeneratorFactory` encapsula a criação dos geradores e recebe `ObjectMapper` via injeção de dependências Spring. `DaoFactory` faz o mesmo para os DAOs, recebendo o `DataSource` correcto conforme o perfil activo (H2, SQLite, PostgreSQL). Os consumers nunca instanciam directamente. (Gamma, Helm, Johnson, & Vlissides, 1994), pp. 107-116.

Data Access Object (Core J2EE Patterns)

`SessionDao`, `ExerciseDao` e `ResultDao` estendem `AbstractDao<T>`, que encapsula a lógica de `Connection`, `PreparedStatement` e `ResultSet`. A camada de serviço nunca escreve SQL. Optei por não usar Spring Data JPA (ADR-010) para manter as queries explícitas e controláveis. (Alur, Crupi, & Malks, 2003), pp. 462-475.

Value Object / Record (DDD)

As entidades de domínio (`Note`, `Interval`, `Scale`, `Chord`) e os DTOs (`SessionRecord`, `ExerciseRecord`, `ResultRecord`) são Java records imutáveis. A imutabilidade elimina bugs de estado partilhado e simplifica o raciocínio sobre o fluxo de dados. (Evans, 2003), pp. 97-103.

Sentinel Value

`MusicConstants.SESSION_NONE = 0L` distingue o modo sandbox (sem persistência) de uma sessão ativa. O campo `sessionId` em `AnswerRequest` é primitivo `long` – quando omitido no JSON, Jackson deserializa para o `0L` automaticamente. Sem nulls, sem validação extra.

2.7 WIREFRAMES

O protótipo de navegação está disponível em [docs/design/wireframes.pdf](#) (versão interactiva em [docs/design/wireframes.html](#)). A interface organiza-se em quatro ecrãs:

- **index.html** - ecrã principal: seletor de tipo de exercício, dois modos de entrada (Praticar sem persistência, ou Iniciar sessão com registo), teclado virtual CSS e modal de ajuda
- **exercise.html** - ecrã de exercício ativo: instrução por tipo, botão de áudio, teclado virtual, feedback após resposta (correto/errado + resposta esperada) **session-end.html** - resumo de sessão: total de exercícios, taxa de acerto, barra de progresso, dica pedagógica da área mais fraca
- **progress.html** - dashboard: precisão global, breakdown por tipo, áreas mais fracas com dicas, histórico de sessões A separação em quatro ecrãs distintos reflecte um princípio pedagógico: o utilizador não deve estar a gerir a interface enquanto tenta ouvir. O ecrã de exercício tem exatamente uma ação possível - ouvir e responder. O dashboard e o resumo de sessão estão fora desse fluxo para não distrair durante a prática.

As decisões de arquitetura do frontend estão documentadas em ADR-017 (deployment: diretoria / `frontend/` servida pelo Spring Boot) e ADR-018 (teclado CSS, range C2-C6 desktop / C3-C5 mobile, breakpoints 768px e 1024px).

2.8 DECISÕES DE ARQUITECTURA

As decisões de arquitectura estão documentadas individualmente em `docs/architecture/adr/`. Esta secção resume cada decisão com as alternativas consideradas e as consequências principais.

ADR-001 - Arquitectura em 3 camadas

A aplicação divide-se em três camadas independentes: Frontend (HTML/JS vanilla), Backend (Java/Spring Boot com API REST) e Persistência (H2 em desenvolvimento). A abordagem monolítica com Thymeleaf foi rejeitada por misturar frontend e backend, dificultando a avaliação independente de cada camada. Microserviços foram rejeitados por introduzir complexidade desproporcional ao âmbito do projeto. A separação facilita desenvolvimento paralelo e extensão futura sem alterar o backend.

ADR-002 - Backend agnóstico à origem do input

O backend recebe respostas musicais sempre pelo mesmo endpoint REST, independentemente de virem do teclado virtual ou de um controlador MIDI físico. Endpoints separados por tipo de input foram rejeitados por violar a separação de responsabilidades - a origem do input é uma preocupação exclusiva da camada de apresentação. O frontend normaliza os eventos heterogéneos para um callback único antes de chamar a API, mantendo o backend simples e extensível para futuros tipos de input.

ADR-003 - Geração procedural de exercícios sem datasets externos

Os exercícios são gerados algoritmicamente a partir do modelo de domínio musical em runtime, sem ficheiros de dados externos nem conteúdo estático pré-definido. Uma base de dados de exercícios pré-definidos foi rejeitada por limitar a variedade e não demonstrar que o modelo musical está corretamente formalizado. APIs externas de teoria musical foram rejeitadas por introduzirem dependências fora do controlo do sistema e potencial violação da restrição CBo3. A abordagem garante variedade virtualmente infinita e deployment simples.

ADR-004 - Web Audio API para reprodução de som

O som é gerado no browser via Web Audio API nativa (Web Audio API — W3C Recommendation, 2021), usando síntese de osciladores com frequência calculada por $f = 440 * 2^{((midiNumber - 69) / 12)}$. Bibliotecas externas como Tone.js e Howler.js foram rejeitadas por introduzirem dependências desnecessárias para o scope. Ficheiros MP3/WAV por nota foram rejeitados por requererem 88 ou mais ficheiros de áudio e violarem a restrição CBo8. A decisão resulta em zero dependências de áudio e controlo total sobre timbre e duração.

ADR-005 - Web MIDI API para input de hardware MIDI

O input de controladores MIDI físicos é capturado via Web MIDI API nativa (Web MIDI API — W3C Editor's Draft, 2015) do browser, com graceful degradation em Safari (único browser moderno sem suporte). Plugins como JazzPlugin foram rejeitados por serem obsoletos e incompatíveis com o objectivo de funcionar no browser sem instalação. Um servidor MIDI via WebSocket foi rejeitado por introduzir um terceiro processo com overhead desproporcional. RFo4 foi classificado como "Should have" para que a ausência de hardware não impeça a entrega.

ADR-006 - Java 21 + Spring Boot para o backend

O backend é implementado em Java 21 com Spring Boot, stack onde tenho experiência directa de desenvolvimento. Node.js, Python/FastAPI e Quarkus foram rejeitados por exigirem curva de aprendizagem onde já existe domínio em Java/Spring. O Spring Boot minimiza configuração via auto-configuration e Tomcat embebido, e o ecossistema extenso garante documentação disponível para qualquer decisão técnica.

ADR-007 - H2 para desenvolvimento; PostgreSQL ou SQLite3 para produção

Em desenvolvimento usa-se H2 (in-memory ou ficheiro), que permite iniciar sem qualquer setup de base de dados e inclui consola web. PostgreSQL desde o início foi rejeitado por exigir instalação antes de escrever a primeira linha de código. A decisão de produção (PostgreSQL vs SQLite3) fica em aberto até antes da demo interna (Olo1); com JDBC puro a troca requer apenas alterar o driver e o schema.sql.

ADR-008 - JavaScript vanilla no frontend (sem frameworks)

O frontend é implementado em HTML e JavaScript ES6+ puro, sem frameworks (React, Vue, Angular) e sem ferramentas de build (npm, Vite, Webpack). React e Vue foram rejeitados por exigirem curva de aprendizagem com JSX, bundler e gestão de estado – complexidade desproporcional ao scope com pouca experiência em frontend. Fico com zero ferramentas de build, código transparente para o júri, e compatibilidade com CBo7.

ADR-009 - Estrutura de ecrãs: páginas HTML separadas

A navegação é feita com páginas HTML separadas servidas diretamente pelo browser, sem router em JavaScript. A abordagem SPA com router em JS puro foi rejeitada por exigir gestão de histórico de browser e estado global - complexidade desnecessária para o nível de experiência atual. SPA com framework viola CBo7. As páginas atuais são index.html, exercise.html, session-end.html e progress.html, cada uma autónoma e testável de forma independente.

ADR-010 - DAO classes com JDBC puro em vez de Spring Data JPA

A camada de persistência usa classes DAO com JdbcTemplate e SQL explícito, sem anotações @Entity, Hibernate, nem JpaRepository. Spring Data JPA foi rejeitado por introduzir lazy loading, proxies e cache - comportamentos difíceis de controlar e de defender perante o júri. O resultado é SQL visível e explícito em cada DAO, com DTOs implementados como Java records imutáveis (Java 16+).

ADR-011 - TDD como metodologia de desenvolvimento

O projeto segue TDD (Red-Green-Refactor) em 85%+ do código de produção: nenhuma feature é implementada sem um teste que falha primeiro, e nenhum bug é corrigido sem o reproduzir em teste. Testes escritos após implementação foram rejeitados por não prevenirem regressões com a mesma eficácia. O critério adotado é que cada contrato funcional da aplicação tem um teste correspondente, focado em comportamento esperado e não em edge cases raros.

ADR-012 - Modelo de dados e estratégia de persistência

O schema de persistência define três tabelas - sessions, exercises e results - com chaves estrangeiras e integridade referencial (CASCADE em sessions, SET NULL em exercises). DAOs sem anotações Spring usam try-with-resources para gestão de conexões, garantindo controlo sobre cada operação. O histórico de resultados suporta dificuldade adaptativa (RF09) via análise dos últimos 100 exercícios do mesmo tipo.

ADR-013 - Schema do campo question nos exercícios

O campo question na tabela exercises armazena JSON mínimo baseado em números MIDI: dois MIDI para intervalos, raiz e tipo para escalas e acordes. Guardar todas as notas da escala ou acorde foi rejeitado por ser redundante - o domínio é determinístico e reconstrói as notas a partir de raiz e tipo. A decisão fecha o open item Olo7 e garante que geradores e controllers usam um schema consistente.

ADR-014 - Protocolo de resposta baseado em notas MIDI

O utilizador responde tocando notas, enviando um array de números MIDI ao endpoint de resposta. O modelo anterior de resposta por nome textual foi rejeitado por reduzir o exercício a escolha múltipla, eliminando a componente de performance musical. As regras de validação variam por tipo: intervalos exigem

MIDI exato, escalas validam padrão de intervalos independente de oitava, acordes validam padrão ascendente I-III-V independente de oitava.

ADR-015 - Taxonomia de dificuldade para exercícios

Foi criado o enum `DifficultyLevel` com 5 bandas semânticas (`BEGINNER` a `EXPERT`) que mapeiam a escala numérica 1-10. Cada tipo de exercício carrega o seu nível e os geradores filtram via `availableFor(DifficultyLevel)`, eliminando *magic numbers*. A alternativa de manter inteiros sem semântica foi rejeitada por tornar `RF09` (dificuldade adaptativa) impossível de implementar de forma correta e consistente. O `DifficultyService` centraliza o algoritmo de adaptação baseado nos últimos 100 resultados por tipo.

ADR-016 - Contrato de Erros: RFC 7807 Problem Details

Todos os erros HTTP usam `ProblemDetail` (RFC 7807 - Problem Details for HTTP APIs, 2016), (nativo no Spring Boot 3) com um `GlobalExceptionHandler` central, eliminando `try-catch` disperso em cada `controller`. A alternativa de endpoints separados por tipo de erro foi rejeitada por violar separação de responsabilidades; manter formatos inconsistentes (plain text, JSON genérico, nada) foi rejeitado por tornar o parsing frontend impossível sem lógica ad-hoc. A centralização garante que todos os clientes HTTP - frontend, Swagger, Postman, integradores - conseguem parsear erros uniformemente e que novos endpoints herdam o comportamento automaticamente.

ADR-017 - Frontend: localização no repositório e integração com Spring Boot

Os ficheiros frontend ficam na diretoria `/frontend/` na raiz do repositório, servida pelo Spring Boot via configuração em `application.properties`. Colocar o frontend em `src/main/resources/static/` foi rejeitado por acoplar a estrutura frontend ao layout Maven.

Repositório separado foi rejeitado por overhead injustificado para MVP académico com um único developer. O URL do backend fica centralizado em `js/api.js` - uma linha para mudar de ambiente em produção.

ADR-018 - Teclado virtual: range MIDI e design responsivo

O teclado virtual é implementado em CSS puro (`flexbox` e `position: absolute` para teclas pretas) com tres breakpoints: mobile (25 teclas C3-C5), tablet (37 teclas C2-C5) e desktop (49 teclas C2-C6). Um piano completo de 88 teclas foi rejeitado por ser inviável em mobile (largura de 2464px). Imagem PNG com image map foi rejeitada por dificultar highlight dinâmico e resize. O range desktop cobre todos os exercícios `BEGINNER` e `ELEMENTARY` sem scroll, com breakpoints alinhados com Bootstrap 5 e Tailwind.

ADR-019 - Expansão do domínio de escalas além do MVP original

O enum `ScaleType` foi construído com 28 tipos de escala em vez dos 3 definidos no MVP, desbloqueados progressivamente por `DifficultyLevel`. Manter apenas os 3 tipos originais foi rejeitado por desperdiçar um modelo já construído sem benefício pedagógico. A expansão é invisível para utilizadores iniciantes (`BEGINNER` vê apenas `MAJOR`) e não exige mudança de código no gerador para adicionar novos tipos. Esta ADR regulariza retroativamente uma decisão implementada sem registo prévio, esta discrepância foi **identificada na auditoria realizada durante a Semana 8**.

ADR-020 - Testes de integração frontend: Cucumber + Selenide

Os testes end-to-end são escritos em Gherkin (.feature files) com Selenide como driver de browser, integrados com `@SpringBootTest(RANDOM_PORT)`. O Gherkin serve duas funções em simultâneo: contrato de teste e documentação dos casos de uso rastreável aos requisitos funcionais. Playwright foi rejeitado por ter menos documentação em contexto académico Java; Cypress foi rejeitado por exigir runtime Node.js separado num repositório Java. JUnit com Selenium puro (sem Cucumber) foi rejeitado por não produzir ficheiros

.feature legíveis por não-programadores. A infraestrutura garante que cada cenário exercita a stack completa - backend real, base de dados H2, e frontend servido pelo classpath - sem mocks.

ADR-021 - Enunciado de intervalo não revela o tipo

O enunciado dos exercícios de intervalo mostra apenas a nota raiz ("Toca este intervalo com raiz em C4"), sem revelar o tipo nem a nota alvo. Mostrar ambas as notas do intervalo foi rejeitado por permitir ao utilizador calcular o tipo por subtração, contornando o objectivo de treino auditivo. O tipo é revelado apenas no campo hint, após submissão. O botão "Ouvir" passa a ser essencial - o utilizador escuta raiz e nota alvo e reproduz por ouvido - o que aumenta a dificuldade percebida mas alinha o exercício com o treino real de identificação de intervalos.

ADR-022 - Raízes de acorde limitadas a teclas brancas em BEGINNER e ELEMENTARY

O gerador de acordes limita as notas raiz a teclas brancas (pitch classes {0, 2, 4, 5, 7, 9, 11} - C D E F G A B) nos níveis BEGINNER e ELEMENTARY; a partir de INTERMEDIATE a gama completa fica disponível. A seleção aleatória de raiz em toda a gama independentemente do nível foi rejeitada por apresentar C# Major ao primeiro exercício de BEGINNER, contradizendo o princípio de progressão pedagógica. O comportamento é consistente com o ScaleExerciseGenerator, que aplica a mesma restrição desde ADR-015. A regra afecta apenas a geração - a validação de respostas não é alterada.

ADR-023 - Presets de timbre Web Audio e scheduling nativo

O teclado virtual oferece quatro presets de timbre seleccionáveis: sine (referência, percepção de altura clara), triangle (mais quente, menos brilhante), sawtooth (brilhante, audível em ambientes ruidosos) e piano (sawtooth com decay rápido, imita percussão de martelo). A escolha persiste via sessionStorage (chave mt_timbre). Amostras de áudio reais foram rejeitadas por violar ADR-003 (geração procedural sem assets externos). Sintetizador FM completo foi rejeitado por complexidade desproporcional ao MVP. O scheduling de notas usa o relógio nativo da Web Audio API (ac.currentTime) em vez de setTimeout, eliminando drops em sequências rápidas causados por throttling do browser.

Capítulo 3 - Implementação

3.1 DECOMPOSIÇÃO EM FASES

O desenvolvimento foi organizado em fases sequenciais, cada uma com pré-condições explícitas. Esta estrutura garantiu que cada fase arrancasse com uma base estável e evitou que bugs de uma fase se propagassem para a seguinte.

Fase	Descrição	Precedência	Estado
0	Setup: Maven, Spring Boot, H2, estrutura de pacotes	-	☑ Completo
1	Modelo de domínio: Note, Interval, Scale, Chord, enums de tipo	Fase 0	☑ Completo
2	Persistência: DTOs, DAOs JDBC, suporte multi-BD, DaoFactory	Fase 1	☑ Completo
3	Geradores de exercícios: Interval, Scale, Chord com Strategy pattern (Gamma, Helm, Johnson, & Vlissides, 1994)	Fase 2	☑ Completo
4	REST API: endpoints de exercícios, sessões, progresso, sandbox	Fase 3	☑ Completo

4.1	RF07/RF08/RF09: persistência de resultados, dashboard, dificuldade adaptativa	Fase 4	☑ Completo
4.2	API polish: RFC 7807, Bean Validation (RFC 7807 - Problem Details for HTTP APIs, 2016), SessionResponse unificado	Fase 4.1	☑ Completo
4.3	Auditoria e correcção de inconsistências	Fase 4.2	☑ Completo
5	Frontend: teclado virtual CSS, Web Audio API (Web Audio API — W3C Recommendation, 2021), index.html	Fase 4	⌚ Pendente
6	Frontend: ecrã de exercício, Web MIDI API (Web MIDI API — W3C Editor's Draft, 2015), fluxo completo	Fase 5	⌚ Pendente
7	Frontend: dashboard, session-end, wireframes implementados	Fase 6	⌚ Pendente
8	Testes de integração frontend-backend, casos limite	Fase 7	⌚ Pendente
9	Revisão final, validação dos critérios de aceitação, capturas de ecrã	Fase 8	⌚ Pendente

Tabela 4 - Decomposição em Fases

3.2 ESTADO DE IMPLEMENTAÇÃO À DATA DO INTERCALAR

Backend - completo

À data deste relatório intercalar (2 de maio de 2026), o backend está completamente implementado e revisto. Todos os requisitos Must-have do MVP estão verificados: geração procedural de exercícios (RF01-RF02), avaliação automática (RF05), persistência de resultados (RF07), dashboard de progresso com identificação de padrões fracos (RF08) e dificuldade adaptativa (RF09). O modo sandbox (RF10) está implementado via o sentinel SESSION_NONE.

São 230 testes a passar - diferentes tipos: testes unitários, de integração e de propriedade (jqwik). A cobertura inclui testes de fronteira para o algoritmo de dificuldade adaptativa (Embretson & Reise, 2000) (exactamente 80%, exactamente 40%, 39%) que estavam em falta antes da auditoria da semana 8.

Na semana 8 foi realizada uma verificação abrangente do backend contra todos os contratos documentados. Foram identificados e corrigidos problemas que a suite de testes não apanhava: o gerador de escalas ignorava o sistema de dificuldade e usava sempre os mesmos 3 tipos, o campo `options` violava o protocolo de verificação da entrada, dois controllers tinham try-catch locais que contornavam o handler de erros central, e o modelo ER tinha inconsistências face ao schema SQL real.

Frontend - desenhado, não implementado

O frontend foi desenhado e especificado: wireframes dos 4 ecrãs produzidos (`docs/design/wireframes.pdf`), decisões de arquitectura documentadas nos ADRs 017 e 018 (deployment, teclado CSS, range MIDI, breakpoints responsivos). A implementação começa na semana 9, após a entrega do relatório intercalar.

Documentação - completa

19 ADRs documentam todas as decisões de arquitectura significativas, com alternativas consideradas e consequências. Os diagramas C4 (contexto e contentores) e o modelo ER estão actualizados e espero que consistentes com o código.

3.3 CALENDÁRIO DA FASE RESTANTE

O calendário revisto para as semanas 9-16, após a entrega intercalar:

Semanas	Datas	Planeado
Sem. 9-10	7-16 mai	Fase 5 - Frontend: teclado virtual CSS, Web Audio API, index.html com selector de tipo e modos Praticar/Sessão
Sem. 11-12	19-30 mai	Fase 6-7 - ecrã de exercício com Web MIDI API, dashboard de progresso, session-end
Sem. 13	2-6 jun	Fase 8-9 - testes de integração, validação de todos os critérios de aceitação do MVP, capturas de ecrã para Cap. 4
Sem. 14	9-13 jun	Cap. 4 (Testes) e Cap. 5 (Conclusões); revisão bibliográfica APA
Sem. 15	16-20 jun	Reunião de preparação para defesa com orientador; ensaio de perguntas
Sem. 16	24 jun	Submissão do relatório final

Tabela 5 - Calendário da Fase Restante

O risco principal para a fase restante é a inexperiência em frontend (Roz nos riscos do projeto). A mitigação está no design minimalista já validado nos wireframes e na decisão de usar vanilla JS sem frameworks, o que reduz a superfície de risco tecnológico, pode acabar por gerar mais código *boilerplate* para realizar ações triviais.

Capítulo 4 - Testes

4.1 FILOSOFIA E ABORDAGEM

Desde o primeiro commit, optei por TDD (Test-Driven Development). Tornei-o obrigatório, e formalizado na ADR-011: nenhum código de produção é escrito sem que exista primeiro um teste a falhar.

A decisão foi simples: num projeto onde a lógica de domínio é, ela própria, teoria musical computada algoritmicamente, a única forma fiável de verificar que "uma Quinta Perfeita a partir de C produz G" é escrever um teste que o afirme explicitamente, e falhar antes de implementar. A cada futura alteração evito regressão ou bugs inesperados, por meio de contratos comportamentais claros. Efeitos colaterais, sobem à superfície rapidamente, em caso de alterações com efeitos inesperados acidentais.

O ciclo RED-GREEN-REFACTOR foi respeitado com commits atômicos e separados: um commit para o teste (RED), um commit para a implementação mínima que o faz passar (GREEN), e um commit opcional de refactor quando o código beneficiava de limpeza sem alterar comportamento. Não posso garantir 100% de aderência, pois que no meio de desenvolvimentos ou por fatores externos, posso ter desviado-me levemente do caminho, mas rapidamente retorno ao contrato. Esta separação deve ser clara e visível no histórico do repositório e serve de documentação implícita da intenção de cada mudança.

4.2 CAMADAS DE TESTE

A estratégia baseia-se em quatro camadas distintas, cada uma com responsabilidade e ferramentas próprias.

Testes de unidade - domínio musical

A camada mais granular cobre os value objects imutáveis (Evans, 2003): Note, Interval, Scale, Chord e os seus respetivos enums. Estes testes verificam propriedades fundamentais - que C4 corresponde ao MIDI 60, que uma Terceira Menor tem 3 semítons, que uma escala Maior de C contém as notas corretas.

Os geradores de exercícios (IntervalExerciseGenerator, ScaleExerciseGenerator, ChordExerciseGenerator) têm cobertura por banda de dificuldade: testa-se que exercícios de nível BEGINNER não incluem intervalos

dissonantes, que raízes em teclas pretas só aparecem a partir de INTERMEDIATE, e que a sequência RED-GREEN confirma o comportamento esperado antes de cada fix.

Testes de integração - persistência

Os DAOs (SessionDao, ExerciseDao, ResultDao) são testados (Alur, Crupi, & Malks, 2003) (Fowler, 2002) contra uma base de dados H2 real - sem mocks. Esta decisão foi intencional e está registada: em minha experiênc pessoal, num projeto anterior de equipa, testes com mocks passaram todos enquanto a migração de produção falhava silenciosamente. Aqui, cada operação de persistência - guardar, atualizar, encontrar por ID, contar por tipo - é verificada com dados reais numa base de dados em memória. Na esperança de simular ambientes com escrita em bases de dados, e execução de SQLs que produzem o resultado esperado.

Testes baseados em propriedades - jqwik

Para verificar que nenhum valor de enum produz comportamento inesperado, recorri a testes baseados em propriedades com a biblioteca jqwik. Em vez de testar casos específicos, estes testes afirmam propriedades que devem ser verdadeiras para qualquer valor possível: "para qualquer IntervalType, o número de semítons está entre 0 e 12", "para qualquer ScaleType, a fórmula de intervalos soma sempre 12 semítons".

Esta abordagem cobriu casos de fronteira que testes manuais dificilmente identificariam - como aliases de modos (IONIAN = MAJOR, AEOLIAN = NATURAL_MINOR) que não devem ser gerados como exercícios independentes.

Validação de qualidade dos testes

A cobertura implícita da pirâmide actua como validador: 156 testes de geradores verificam combinações dificuldade × tipo × raiz para intervalos, escalas e acordes. Um gerador que produzisse valores fora do contrato seria detectado imediatamente por qualquer uma dessas 156 combinações.

Mutation testing via PITest foi avaliado durante o desenvolvimento e removido: o JUnit Platform engine do Cucumber descobre testes via cucumber.features em junit-platform.properties e o PITest trata os cenários Gherkin como testes de unidade das classes de domínio. Na fase de pré-scan, o PITest tenta executar cada cenário E2E isoladamente — sem servidor Spring Boot, sem Chrome — e aborta antes de chegar às mutações. Derivado de conflitos que demandariam mais tempo do que trariam benefícios neste curto espaço de tempo, abandonei a ideia do PITest para este projeto, podendo ser retomada numa futura interação. Acredito que a cobertura de qualidade é assegurada pelos testes jqwik, que cumprem o mesmo propósito com uma abordagem mais adequada ao modelo de testes deste projeto.

Testes E2E - Cucumber e Selenide

A camada mais externa cobre o sistema completo: browser real (Chrome headless), servidor Spring Boot numa porta aleatória, e frontend servido diretamente pelo classpath. Os testes são escritos em Gherkin (.feature files) que servem simultâneamente como especificação executável e documentação de comportamento.

Os cenários cobrem o fluxo completo do utilizador: abrir a aplicação, seleccionar tipo de exercício e modo, interagir com o teclado virtual, submeter resposta, ver feedback, e verificar persistência no dashboard de progresso. A infraestrutura usa a ADR-020 (Cucumber + Selenide) e foi desenvolvida após os desafios de um contexto headless - especificamente, que o AudioContext do Web Audio API requer interação do utilizador para arrancar, o que levou a uma correcção defensiva no handleNoteOn.

4.3 MÉTRICAS

No estado final do projeto, a suite de testes compreende:

Camada	Ferramenta	Testes
Unidade e integração	JUnit 5 + AssertJ	342
Baseados em propriedades	Jqwik	incluídos nos 342
E2E (browser completo)	Cucumber + Selenide	100
Total		442

Tabela 6 - Estado final suite de testes

Todos os testes passam na CI local (`mvn test -Ddb.type=H2`), incluindo os E2E em modo headless. O tempo total de execução da suite completa é de aproximadamente 4-5 minutos, dominado pelos testes E2E que iniciam um browser Chrome por cada cenário.

Os 342 testes unitários distribuem-se por: domínio musical (78), geradores de exercícios (156, maioritariamente `ChordExerciseGeneratorTest` com 112 combinações dificuldade × tipo × raiz), serviços (44), controladores REST (36), persistência DAO (14), e configuração (7+1).

4.4 O QUE OS TESTES NÃO COBREM (E PORQUÊ)

A síntese de áudio via Web Audio API não é coberta por testes automatizados. Testei manualmente os quatro presets de timbre (sine, triangle, sawtooth, piano) em diferentes oitavas e confirmei que o scheduling nativo reduz significativamente os drops em input rápido. Contudo, verificar que um oscilador produz a frequência correta em ambiente headless exigiria ferramentas de análise de áudio que excedem o âmbito deste projeto.

Da mesma forma, a integração com controladores MIDI físicos foi testada manualmente com um teclado USB - a Web MIDI API não está disponível em todos os browsers nem em ambiente headless.

4.5 COBERTURA DA BASE DE DADOS SQLITE

Os testes DAO correm contra H2 em memória por defeito. Para verificar que a configuração de produção - SQLite3 com `db.sqlite.path` - funciona do início ao fim, existe `SqliteEndToEndIT`: inicia o servidor com perfil SQLite, aponta para um ficheiro temporário, corre o fluxo completo (iniciar sessão, gerar exercício, submeter resposta, terminar, verificar progresso no dashboard), e remove o ficheiro no teardown.

Este teste foi o que confirmou que `db.sqlite.path` com caminho absoluto funciona em runtime - não apenas nos testes unitários de `DatabaseFactory`. SQLite com caminhos relativos comporta-se de forma diferente do H2 e só se detecta com um ficheiro real no filesystem.

4.6 RESULTADOS DE PERFORMANCE

O requisito RNFo6 estabelece um limite de zooms para a validação de respostas em ambiente local. Medi os cinco endpoints principais com dez pedidos cada, usando curl com timing HTTP completo (connect + transfer), em loopback (latência de rede excluída).

Endpoint	Método	Média (ms)	Mín (ms)	Máx (ms)
<code>/api/exercises/generate</code>	POST	12.5	8.0	28.7

/api/progress	GET	8.7	5.2	14.9
/api/sandbox/note/60	GET	3.6	2.9	5.6

Tabela 7 - Resultado Testes Performance de Endpoints

Nenhum endpoint excedeu 29ms. O pior caso observado foi 28.7ms, seis vezes abaixo do limite de 200ms. A variância é típica de JVM com warm-up: os primeiros pedidos são sistematicamente mais lentos que os subsequentes.

Os resultados completos estão em docs/architecture/performance-results.md.

Capítulo 5 - Conclusões

5.1 O QUE FOI CONSTRUÍDO

O Musical Theory Trainer que existe no final deste projeto é substancialmente mais rico do que o MVP inicialmente proposto.

5.1.1 Artefactos do projeto

<https://github.com/jrzabott/21184-ProjetoLEI>

5.1.2 Backend

Funcionalidades: Geração procedural de exercícios, avaliação por notas MIDI, persistência com suporte a três bases de dados, dificuldade adaptativa.

Sobre a dificuldade adaptativa: foi implementado durante as primeiras semanas, com 230 testes a passar antes do relatório intercalar.

5.1.3 Frontend

Teclado virtual de duas oitavas e meia (C2-C6), quatro *presets* de síntese de áudio, seletor de timbre, feedback imediato, dashboard de progresso, suporte a controladores MIDI físicos - foi completado na segunda fase do projeto (após relatório intercalar).

5.1.4 Milestones Concretas

A aplicação corre, persiste dados entre sessões, avalia respostas para intervalos, escalas e acordes, adapta a dificuldade com base no histórico do utilizador, e identifica as áreas mais fracas para orientar a prática.

5.2 OBJETIVOS VS. RESULTADO

O documento de proposta estabeleceu, com método MoSCoW, um conjunto de catorze requisitos funcionais (RF01-RF14) e nove não funcionais (RNFO1-RNFO9). Os *Must Have* eram o mínimo aceitável, os *Should Have* eram o que esperava atingir, e os *Could Have* ficaram deliberadamente para depois do MVP estar estável.

Todos os *Must Have* foram cumpridos. Os *Should Have* foram implementados - dificuldade adaptativa (RF09), dashboard de progresso (RF08), suporte MIDI (RF04, RF12) e a integração backend agnóstica ao input (RF13).

O único requisito que ficou por implementar foi o RF14 (mnemónicos de intervalos), que era explicitamente *Could Have*. A decisão de diferir foi documentada como Olo2: há uma questão de direitos de autor por resolver relativamente às associações mnemónicas mais conhecidas (por exemplo, "Twinkle Twinkle" para a

5ª Perfeita). Optei por não incluir o feature em vez de o incluir com material de origem duvidosa e base suficiente para justificar e defender a utilização.

5.2.1 Dificuldade Adaptativa

A dificuldade adaptativa merece um parágrafo à parte, porque é o requisito com mais implicações pedagógicas e onde existe mais literatura disponível para apoiar - ou contestar - a decisão tomada.

Há pelo menos quatro abordagens distintas.

1. O modelo Elo (Elo, 1978), concebido para classificação em xadrez e adotado pelo Duolingo, atribui uma pontuação de dificuldade aos exercícios e uma pontuação de capacidade ao utilizador, ajustando ambas em cada resposta.
2. O Item Response Theory (Embretson & Reise, 2000), padrão em psicometria e exames normalizados, estima a probabilidade de resposta correta a partir da capacidade latente do utilizador e dos parâmetros de cada item - mais rigoroso matematicamente, mas requer grandes volumes de dados de calibração que este projeto não tem.
3. A repetição espaçada (Ebbinghaus, 1964) (Wozniak & Gorzelańczyk, 1994), base do Anki, otimiza intervalos de revisão para maximizar retenção a longo prazo: é um objetivo diferente, mais focado na memória do que no desafio progressivo. ‘
4. E o modelo de limiar simples, que é o que está implementado: taxa de acerto acima de 80% aumenta a dificuldade, abaixo de 40% reduz, com janela deslizante de 100 exercícios.

A escolha do limiar simples não foi por omissão das alternativas. Vygotsky descreveu a *Zone of Proximal Development* como a distância entre aquilo que o aprendiz consegue fazer de forma autónoma e aquilo que consegue fazer com orientação (Vygotsky, 1978). O ponto de aprendizagem mais produtivo é esse espaço - nem aquém (o material já foi integrado, não há crescimento), nem além (está fora de alcance, o que gera frustração, não progresso). O limiar de 80% como fronteira superior operacionaliza exatamente esta ideia: quando o utilizador acerta de forma consistente em 4 de cada 5 exercícios de um dado nível, esse nível saiu da zona proximal. O limiar de 40% como fronteira inferior assinala o oposto - o material está a ser proposto fora do alcance e continuar sem ajustar não é desafio, é atrito. Csikszentmihalyi (1990) descreve este equilíbrio entre desafio e capacidade como condição necessária para o estado de flow - o ZPD de Vygotsky é, em termos pedagógicos, a mesma fronteira. (Csikszentmihalyi, 1990)

A janela de 100 exercícios foi pensada para resistir ao ruído: uma série de erros numa sessão de concentração baixa não deve relegar o utilizador para um nível mais fácil de forma permanente. A janela distribui o peso histórico de forma que uma sessão má não cancela o progresso acumulado.

O que o modelo não faz - e assumo essa limitação - é calibrar empiricamente os limiares. Os valores de 80% e 40% derivam de literatura pedagógica (Bloom, 1984), apresenta limiares semelhantes no contexto de mastery learning), mas não foram validados com utilizadores reais desta aplicação. Num contexto de investigação seria expectável um ciclo de validação com estudantes. Este projeto não tem esse ciclo; O que este projeto tem, em vez do ciclo de validação é a justificação teórica e a transparência sobre o que ficou por fazer.

Há também objetivos pedagógicos deliberadamente fora do âmbito:

- O treino auditivo - ouvir um intervalo e identificar o tipo pelo som - é o complemento natural da teoria que a aplicação treina, mas é um problema inverso e uma competência diferente (pelo menos não coberta pela aplicação).
- A leitura de partituras, o ritmo e a métrica ficaram fora do MVP por decisão de manter o *scope* consciente, sendo esta registada na proposta.
- O que talvez merecesse um pouco mais atenção, e não recebeu: contexto harmónico:
 - um Trítone no vazio é diferente pedagogicamente do mesmo Trítone numa tensão dominante que resolve para a tónica.

- A aplicação trata ambos como equivalentes porque o requisito era identificação isolada de intervalos, não a sua função dentro de uma progressão.
- Isso é uma limitação real dentro do âmbito do projeto e não apenas uma omissão por descuido.

5.2.2 Added Value – Quality of Life Improvements

Há algumas pequenas funcionalidades que considero *quality of life improvements* que não estavam previstas na proposta e foram construídas de qualquer forma e merecem nota:

1. a infraestrutura de testes E2E com Cucumber e Selenide (96 cenários a correr em Chrome headless)
2. o suporte a múltiplas bases de dados via Strategy + Factory (H2 em dev, SQLite3 em produção, com extensão para PostgreSQL sem alterar uma linha de lógica de negócio) (Gamma, Helm, Johnson, & Vlissides, 1994)
3. o seletor de timbre com quatro presets de síntese Web Audio.

Estes itens não foram pedidos - surgiram como consequência de querer que o sistema fosse testável e sustentável e minimamente divertido (e variado) aquando da utilização em âmbito de desenvolvimento.

Tabela de Requisitos Funcionais

ID	Descrição resumida	MoSCoW	Estado	Notas
RF01	Geração procedural de exercícios sem datasets externos	Must	☑	Intervalos, escalas, acordes via algoritmo determinístico
RF02	Cobertura: notas/oitavas, intervalos, escalas, tríades	Must	☑	18 intervalos, 10+ escalas, 4 tipos de tríade
RF03	Teclado virtual clicável (rato)	Must	☑	C2-C6, 4.5 oitavas
RF04	Input via controlador MIDI físico	Should	☑	Web MIDI API, detecção automática
RF05	Avaliação automática com feedback imediato	Must	☑	< 200ms em ambiente local (ver secção 4.6)
RF06	Feedback sonoro distinto: acerto vs erro	Must	☑	Web Audio API, sons distintos
RF07	Histórico de respostas e sessões	Must	☑	SQLite3 em produção, persistência entre sessões
RF08	Dashboard de progresso	Should	☑	Taxa por tipo, padrões fracos, histórico de sessões
RF09	Dificuldade adaptativa	Should	☑	80%/40%, janela de 100 exercícios, ADR-015
RF10	Modo sandbox	Could	☑	index.html - identificação de notas e intervalos em tempo real

RF11	Teclado reproduz som	Must	☑	Web Audio API, 4 presets
RF12	Detecção automática de MIDI	Should	☑	LED de status, sem configuração
RF13	Backend agnóstico ao input	Must	☑	ADR-002: mesmo endpoint independente da origem
RF14	Mnemónicos de intervalos	Could	✗	OIO2: direitos de autor não verificados

Tabela 8 - Requisitos Funcionais

Tabela de Requisitos Não Funcionais

ID	Descrição resumida	MoSCoW	Estado	Notas
RNF01	Java 21 + Spring Boot + JDBC puro	Must	☑	ADR-006, ADR-010
RNF02	Frontend HTML + JS ES6+ vanilla, sem frameworks	Must	☑	ADR-008
RNF03	Áudio via Web Audio API nativa	Must	☑	ADR-004
RNF04	MIDI via Web MIDI API nativa	Must	☑	ADR-005
RNF05	H2 em dev, SQLite3/PostgreSQL em produção	Must	☑	ADR-007, ADR-015, SQLite3 escolhido (CBo2)
RNF06	Validação < zooms em ambiente local	Must	☑	Medido (ver secção 4.6)
RNF07	Legibilidade académica; Javadoc nos value objects	Must	☑	Classes de domínio com Javadoc descrevendo conceito musical
RNF08	Git com Conventional Commits; commits regulares	Should	☑	~400 commits, histórico contínuo desde março 2026
RNF09	TDD: código de produção precedido por teste falhante	Must	☑	ADR-011; evidência no histórico git (RED/GREEN/REFACTOR)

Tabela 9 - Requisitos Não Funcionais

Algumas breve reflexões sobre os itens *extras*: se a aplicação precisava de ser testável em condições reais, a infraestrutura de testes tinha de existir; se a base de dados ia mudar entre ambientes, a arquitectura tinha de suportar isso sem intervenção direta no código. A consequência é que o ~~protótipo~~ produto entregue tem mais profundidade do que o MVP proposto, mas não se perdeu o fio aos requisitos originais.

5.3 O QUE APRENDI A FAZER DIFERENTE

TDD muda a relação com o código. Escrever o teste antes da implementação não é apenas uma técnica - é uma forma diferente de pensar o problema. Quando a primeira ação é formular "o que é que este código deve fazer e como verifico isso?", as interfaces ficam mais claras, os casos de fronteira aparecem cedo, e a implementação emergente tende a ser mais simples (apesar de casos que requerem abstrações, acabam por ser mais complexos) do que a que eu teria escrito de outra forma. Não foi sempre confortável - há momentos em que a vontade de escrever código antes do teste é forte - mas o histórico de commits (RED-GREEN-refactor) funciona como um *dopamine shot* e acaba por disciplinar e ser um escape ao trabalho implícito de montar um quebra-cabeças, temos pequenos momentos de satisfação quando testes começam a passar. Este *boost* de motivação faz grande diferença. Outra vantagem, o TDD faz com que certas abstrações ou "delimitações de fronteiras" façam-se claras logo ao princípio, durante as primeiras reflexões. Muitas funcionliades ou alto acoplamento logo manifestam-se se estivermos atentos.

Padrões de design não são ornamento académico, ou apenas estética. O Strategy + Factory para suporte a múltiplas bases de dados (Gamma, Helm, Johnson, & Vlissides, 1994) é o exemplo mais claro desde prprojeto: quando surgiu a necessidade de testar com SQLite real em vez de H2, a troca foi uma propriedade de configuração, sem alterar uma linha de lógica de negócio. O mesmo para os geradores de exercícios: adicionar um novo tipo de exercício é adicionar uma classe que implementa `ExerciseGenerator`, registá-la no factory, e o resto do sistema adapta-se.

O frontend é mais difícil de testar do que parece. Descobri durante o code review que o `AudioContext` do Web Audio API lança uma exceção em Chrome headless (Web Audio API — W3C Recommendation, 2021) por falta de interação do utilizador, o que impedia que as notas ficassem registadas na fila de exercícios. A correção (um try-catch defensivo em `handleNoteOn`) resolveu o problema, mas a descoberta foi possível apenas porque os testes E2E estavam a correr numa pipeline real com Chrome headless - sem eles, este bug teria chegado à entrega final.

5.4 LIMITAÇÕES

A aplicação foi testada em Chrome e Edge. Firefox tem suporte limitado à Web MIDI API e comportamentos diferentes no Web Audio (Web MIDI API — W3C Editor's Draft, 2015). Foi breve e superficialmente testada em dispositivos móveis, onde o teclado virtual pode não ser usável sem adaptações de CSS.

A dificuldade adaptativa usa uma janela de 100 exercícios e limiares de 80%/40% - valores derivados de literatura pedagógica mas não calibrados empiricamente com utilizadores reais. Num produto em produção, estes parâmetros mereciam um ciclo de validação com estudantes.

A síntese de áudio, mesmo com os quatro presets, é sintética. Piano, sine, triangle e sawtooth são reconhecíveis mas distantes de instrumentos reais. Para um uso pedagógico sério, amostras de piano real seriam mais eficazes - mas contrariavam a ADR-003 (geração procedural, sem assets externos).

5.5 TRABALHO FUTURO

O que ficou de fora e faria sentido numa versão seguinte:

Modo de identificação de intervalos. A aplicação pede ao estudante para tocar um intervalo dado o tipo e a raiz. O inverso - ouvir um intervalo e identificar o tipo - seria o complemento natural para treino auditivo. Esta divisão foi deliberada e documentada: teoria musical por um lado, *ear training* pelo outro.

Exportação de progresso. O dashboard mostra a evolução mas não permite exportar dados. Para um professor acompanhar múltiplos alunos, seria útil um relatório em CSV ou PDF.

Modo dois jogadores. Dois estudantes a competir em tempo real, com o mesmo exercício em simultâneo. Exigiria WebSockets mas o modelo de dados de sessão já suportaria a extensão com poucos campos adicionais.

Afinação por temperamento. Atualmente, a síntese usa temperamento igual (440Hz como referência). Suporte a temperamento justo ou histórico seria relevante para estudantes de música antiga.

Suporte a autenticação. A aplicação não contempla a utilização de múltiplos utilizadores, nem o registo em separado de diferentes utilizadores. Esta limitação foi intencionalmente escolhida para que o foco do projeto fosse nos itens de proposta inicial, evitando prender-me em *opsec*, complexidades de autenticação, gestão de utilizadores, backend com uma visão admin entre outras derivações naturais de ambientes com múltiplos utilizadores.

5.6 Nota final

Este projeto foi construído num contexto pessoal que tornava a execução mais difícil do que o normal. O facto de estar funcionalmente completo, com uma suite de testes extensa, uma arquitetura limpa e documentação que reflecte as decisões reais - não apenas as que ficariam bem no relatório - é algo de que me orgulho genuinamente. Evitei fortemente os *happy accidents* e trabalhei em cima de cada funcionalidade para que estas fossem divertidas (durante a fase de desenvolvimento) mas também fáceis de utilizar da perspectiva de um utilizador.

A teoria musical é uma linguagem. Como qualquer linguagem, aprende-se melhor quando há feedback imediato e oportunidade de repetição com variação. Como na teoria musical, excepto pelo *devil's interval* não há muito calão que fomente a aprendizagem rápida, como o famoso ditado sobre novos idiomas sugere. Penso que esta ferramenta oferece isso. O resto depende de quem a usa e da consistência com que a usa - que é, no fundo, o que define qualquer prática.

Bibliografia

- Alur, D., Crupi, J., & Malks, D. (2003). *Core J2EE Patterns: Best Practices and Design Strategies*. Prentice Hall.
- Bloom, B. (1984). The 2 Sigma Problem: The Search for Methods of Group Instruction as Effective as One-to-One Tutoring. *Educational Researcher*.
- Claude Code [ferramenta IA]. (2025). Obtido de <https://www.anthropic.com>
- Csikszentmihalyi, M. (1990). *Flow: The Psychology of Optimal Experience*. Harper and Row.
- Ebbinghaus, H. (1964). *Memory: A Contribution to Experimental Psychology*. Dover Publications.
- Elo, A. (1978). *The Rating of Chessplayers, Past and Present*. Arco Publishing.
- Embretson, S., & Reise, S. (2000). *Item Response Theory for Psychologists*. Lawrence Erlbaum Associates.
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- RFC 7807 - Problem Details for HTTP APIs. (2016). Obtido de <https://tools.ietf.org/html/rfc7807>

Spring Boot 3.3.0 Reference Documentation. (2024). Obtido de <https://docs.spring.io/spring-boot/docs/3.3.0/reference/html/>

Vygotsky, L. (1978). *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press.

Web Audio API — W3C Recommendation. (2021). Obtido de <https://www.w3.org/TR/webaudio/>

Web MIDI API — W3C Editor's Draft. (2015). Obtido de <https://www.w3.org/TR/webmidi/>

Wozniak, P., & Gorzelańczyk, E. (1994). Optimization of Repetition Spacing in the Practice of Learning. *Acta Neurobiologiae Experimentalis*.